

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Ingeniería en Sistemas de Información

Arquitectura de Computadores II – Sección 040

Catedrático: [Nombre del catedrático]

Centinela Urbano: diseño, justificación y simulación de un sistema de monitoreo perimetral urbano basado en una arquitectura distribuida orientada a eventos

Proyecto final

Integrantes del grupo [número]

[Nombre completo del integrante 1]	Carné 0000-00-00000
[Nombre completo del integrante 2]	Carné 0000-00-00000
[Nombre completo del integrante 3]	Carné 0000-00-00000
[Nombre completo del integrante 4]	Carné 0000-00-00000
[Nombre completo del integrante 5]	Carné 0000-00-00000

Guatemala, 31 de octubre de 2026

Índice

1. Introducción 1
2. Objetivos 1
    2.1 Objetivo general 1
    2.2 Objetivos específicos 2
3. Escenario de negocio y alcance 2
    3.1 Cliente hipotético 2
    3.2 Alcance de la simulación 3
4. Marco conceptual 4
    4.1 Arquitectura orientada a eventos 4
    4.2 Procesamiento de flujos en tiempo real 5
    4.3 Procesamiento de eventos complejos y correlación 5
    4.4 Bases de datos de series de tiempo 6
    4.5 Privacidad y gobernanza de datos personales 6
    4.6 Objetivos de nivel de servicio 6
    4.7 Visualización para la toma de decisiones 6
5. Arquitectura propuesta 7
    5.1 Visión general y flujo de eventos 7
    5.2 Infraestructura, nodos y segmentos de red 9
    5.3 Componentes y responsabilidades 12
    5.4 Modelo de correlación e inferencia 13
    5.5 Visualización y consola del operador 16
6. Justificación técnica de las decisiones 20
    6.1 Plataforma de despliegue (ADR-001) 22
    6.2 Broker y persistencia (ADR-002 y ADR-003) 23
    6.3 Tareas periódicas y contenedores (ADR-004 y ADR-005) 23
    6.4 Resiliencia, seguridad y observabilidad (ADR-006 a ADR-008) 24
    6.5 Estilo arquitectónico y lenguaje (ADR-010) 25
7. Evaluación de resultados 26
    7.1 Metodología 26

7.2 Validación de correlaciones esperadas.....27
7.3 Análisis de falsos positivos.....28
7.4 Integración, trazabilidad y visualización.....30
7.5 Comportamiento ante picos de carga.....31
7.6 Resiliencia y distribución de carga.....32
7.7 Hallazgos y mejoras.....35
8. Comparación con casos reales de ciudades inteligentes.....37
9. Conclusiones.....39
10. Recomendaciones y trabajo futuro.....40
Referencias.....41
Anexos.....43

1. Introducción

Las ciudades latinoamericanas enfrentan el reto de responder con rapidez a incidentes de seguridad ciudadana con recursos limitados. Cada vez más municipalidades instalan cámaras, botones de pánico y aplicaciones de reporte, pero la información llega fragmentada: un botón de pánico aislado puede ser un error, un exceso de velocidad ocurre a diario y un sonido fuerte puede ser un cohete. El valor no está en cada sensor por separado, sino en la capacidad de **correlacionar** señales heterogéneas que, ocurriendo en el mismo lugar y en el mismo intervalo de tiempo, permiten inferir que algo grave está sucediendo.

Este informe presenta **Centinela Urbano**, el diseño, la justificación y la simulación funcional de un sistema de monitoreo perimetral urbano basado en una arquitectura distribuida orientada a eventos. El sistema emula cinco tipos de sensores urbanos (botones de pánico, cámaras de lectura de placas, sensores de velocidad, sensores acústicos y reportes ciudadanos), procesa sus eventos en tiempo real, los correlaciona para inferir incidentes como un robo en curso o un accidente vial y los presenta a los operadores en una consola en vivo y en tableros de Grafana.

Conforme a lo solicitado, el grupo actuó como un equipo de arquitectura de software: cada decisión relevante quedó documentada como un *Architecture Decision Record* (ADR), siguiendo el formato propuesto por Nygard (2011) y ampliado con el impacto en el negocio para un público no técnico. Los diez ADRs se consolidan en el Anexo A. El documento describe el escenario de negocio elegido, la arquitectura propuesta, la justificación técnica de cada decisión con referencia a los ADRs, la evaluación de los resultados obtenidos en la simulación y una comparación con casos reales de ciudades inteligentes.

2. Objetivos

2.1 Objetivo general

Diseñar, justificar y desarrollar una simulación funcional de un sistema de monitoreo perimetral urbano con capacidades de detección, procesamiento, correlación

y visualización de eventos críticos, basado en una arquitectura distribuida y respaldado por decisiones de arquitectura documentadas y defendibles ante actores no técnicos.

2.2 Objetivos específicos

1. Definir un escenario de negocio concreto y seleccionar el modelo de despliegue adecuado, con estimación de costo y de latencia extremo a extremo.
2. Simular de forma lógica y programática los cinco tipos de sensores urbanos del enunciado, incluyendo picos de carga y escenarios reproducibles.
3. Implementar un modelo de correlación e inferencia explicable que distinga incidentes reales de posibles falsos positivos.
4. Proteger los datos personales (placas y ubicaciones) mediante técnicas de seudonimización, minimización y retención limitada.
5. Definir objetivos de nivel de servicio medibles y la respuesta automática del sistema ante su incumplimiento.
6. Evaluar el sistema bajo carga normal, picos y fallas, y compararlo con experiencias reales de ciudades inteligentes.

3. Escenario de negocio y alcance

3.1 Cliente hipotético

El enunciado exige declarar qué tipo de cliente se atiende, porque la arquitectura correcta depende del caso de negocio. Se definió como cliente una **municipalidad de tamaño medio del área metropolitana de Guatemala**, con unos 500 000 habitantes divididos en 12 zonas, un centro de monitoreo municipal que opera 24 horas y un presupuesto de inversión limitado. Sus prioridades son tres: responder rápido a emergencias, mantener bajo su control los datos personales de la ciudadanía y operar con un gasto predecible, sin depender de facturación variable en dólares ni de la disponibilidad del enlace a internet para detectar incidentes.

3.2 Alcance de la simulación

Como establece el enunciado, la solución no utiliza dispositivos físicos. Un componente de software (*sensor-simulator*) emula el comportamiento, la frecuencia, la ubicación y el tipo de evento de cada sensor. La ciudad simulada tiene 12 zonas. Para dibujarlas en el mapa se tomaron como referencia cartográfica los centroides aproximados de las zonas 1 a 12 de la Ciudad de Guatemala, y un evento que trae coordenadas sin nombre de zona se asigna a la zona más cercana. La Tabla 1 resume los sensores simulados, sus datos y su frecuencia en operación normal.

Tabla 1

Sensores urbanos simulados y datos generados

Sensor	**Datos generados (campos del enunciado)**	**Inventario por zona**	**Frecuencia normal (12 zonas)**
Botón de pánico	timestamp, zona_geografica, tipo_alerta, identificador_ dispositivo, user_context	5 postes + usuarios de la app	≈ 43 por hora
Cámara LPR	placa_vehicular, ubicacion_ sensor, velocidad_estimada, timestamp, modelo y color	3 cámaras	7,2 por segundo
Sensor de velocidad	velocidad_detectada, zona, timestamp, sensor_id, dirección, contravía	4 sensores	6,0 por segundo
Sensor acústico	tipo_sonido_detectado, nivel_decibeles, zona, timestamp, probabilidad_ evento_critico	2 sensores	0,24 por segundo
Reporte ciudadano	tipo_evento, mensaje_ descriptivo, ubicacion_	App y puntos físicos	≈ 86 por hora

Sensor	Datos generados (campos del enunciado)	Inventario por zona	Frecuencia normal (12 zonas)
	aproximada, origen, timestamp		

Nota. El total en operación normal es de 13,5 eventos por segundo. Los picos se simulan multiplicando las tasas (×20 = 270 eventos por segundo). Elaboración propia.

El simulador genera tráfico de fondo con distribuciones semirrealistas: velocidades normales alrededor del límite de cada sensor, anomalías poco frecuentes y sonidos críticos de probabilidad moderada (en Guatemala la pirotecnia provoca falsos disparos con frecuencia). Además ofrece diez escenarios reproducibles que corresponden a los casos de uso de la sección 7 del enunciado (robo en curso, falso positivo, accidente vial, disparos, incendio, vehículo robado, botón abusivo, emergencia médica, pico de carga y el ejemplo exacto de la tabla 2.4), la reproducción de eventos desde archivos CSV con aleatorización y un buffer en disco para no perder eventos críticos. Para que el resultado de un escenario no dependa del azar, mientras se ejecuta sus zonas quedan aisladas del tráfico de fondo y comienzan sin historial ni incidentes abiertos. El operador también puede enviar reportes ciudadanos con texto libre y botones de pánico por la misma puerta HTTP que usaría la aplicación móvil.

4. Marco conceptual

4.1 Arquitectura orientada a eventos

En una arquitectura orientada a eventos los componentes no se invocan directamente: publican hechos que ya ocurrieron y otros componentes reaccionan a ellos. Kleppmann (2017) describe cómo los registros de eventos (*logs*) distribuidos permiten desacoplar a los productores de los consumidores, de modo que cada consumidor procese los datos a su propio ritmo y que nuevos consumidores puedan incorporarse sin modificar a los existentes. Apache Kafka nació precisamente como un sistema de mensajería distribuido para el procesamiento de registros de alto volumen

(Kreps et al., 2011) y organiza los mensajes en *topics* divididos en particiones ordenadas que se reparten entre los consumidores de un mismo grupo (The Apache Software Foundation, s. f.).

4.2 Procesamiento de flujos en tiempo real

El procesamiento por lotes acumula datos y los analiza después; el procesamiento de flujos, en cambio, trata cada evento en cuanto llega, sin esperar a que se complete un conjunto (Kleppmann, 2017). Stonebraker et al. (2005) resumieron en ocho requisitos lo que debe cumplir un sistema de este tipo, entre ellos mantener los datos en movimiento sin pasar por una base de datos en el camino crítico, tolerar datos tardíos, faltantes o desordenados, producir resultados predecibles, integrar los datos en vivo con los almacenados, garantizar la disponibilidad de los datos ante fallas, repartir el procesamiento entre máquinas y responder de inmediato. En este proyecto esos requisitos se traducen en decisiones concretas: el motor de correlación trabaja sobre el estado en memoria de Redis y la base de datos queda fuera del camino de la alerta, las ventanas se evalúan por la hora del evento, los eventos críticos se guardan en disco antes de enviarse, el estado vive fuera de cada instancia para que Kafka pueda repartir las particiones y la latencia se mide contra un objetivo de dos segundos.

4.3 Procesamiento de eventos complejos y correlación

El procesamiento de eventos complejos (CEP, por sus siglas en inglés) busca derivar conocimiento de alto nivel a partir de patrones entre múltiples eventos simples (Luckham, 2002). Etzion y Niblett (2010) identifican como elementos centrales los contextos temporales (ventanas de tiempo), los contextos espaciales y los patrones que combinan eventos de distintas fuentes. En este proyecto, el contexto espacial es la zona de la ciudad, el contexto temporal es una ventana alrededor del evento que inicia la sospecha y el patrón es una combinación ponderada de evidencias. Un aspecto crítico en sistemas distribuidos es la diferencia entre la hora en que ocurre un evento y la hora en que se procesa: correlacionar por la hora del evento permite tolerar llegadas desordenadas (Kleppmann, 2017).

4.4 Bases de datos de series de tiempo

Los eventos de sensores son datos de series de tiempo: se escriben continuamente, casi nunca se modifican y se consultan por rangos temporales. Las bases de datos especializadas particionan los datos por tiempo, los comprimen y aplican políticas de retención automáticas. TimescaleDB implementa estas capacidades como una extensión de PostgreSQL, lo que permite combinar series de tiempo con tablas relacionales y SQL estándar (Timescale, s. f.).

4.5 Privacidad y gobernanza de datos personales

Las placas vehiculares y las ubicaciones permiten reconstruir los desplazamientos de las personas. En Guatemala, la Ley de Acceso a la Información Pública (Congreso de la República de Guatemala, 2008) regula los datos personales sensibles en poder de los sujetos obligados, entre ellos las municipalidades, y establece el *habeas data*. Como referencia internacional, el Reglamento General de Protección de Datos europeo consagra los principios de minimización de datos, limitación del plazo de conservación y protección de datos desde el diseño, y reconoce la seudonimización como una medida técnica de protección (Parlamento Europeo y Consejo de la Unión Europea, 2016).

4.6 Objetivos de nivel de servicio

La ingeniería de confiabilidad distingue entre indicadores de nivel de servicio (SLI), que miden un aspecto concreto del servicio, y objetivos de nivel de servicio (SLO), que fijan la meta para ese indicador (Beyer et al., 2016). Un SLO solo es útil si se acompaña de una respuesta definida cuando se incumple, que es exactamente lo que exige el enunciado al pedir qué ocurre más allá de tener tableros en Grafana.

4.7 Visualización para la toma de decisiones

En un centro de monitoreo, la visualización tiene una función precisa, que es sostener la conciencia situacional del operador. Endsley (1995) la describe en tres niveles: percibir los elementos del entorno, comprender su significado y proyectar su estado próximo. Una pantalla que muestra cada lectura por separado satura la atención y

la memoria de trabajo, que son justamente los factores que limitan esa conciencia; una que presenta incidentes ya correlacionados, con su severidad, su ubicación y la evidencia que los explica, permite comprender y decidir en segundos. Por esa razón el sistema separa la consola del operador, centrada en alertas explicadas que se validan con un clic, de los tableros de Grafana, que muestran el mapa de la ciudad, las tendencias y la salud del sistema.

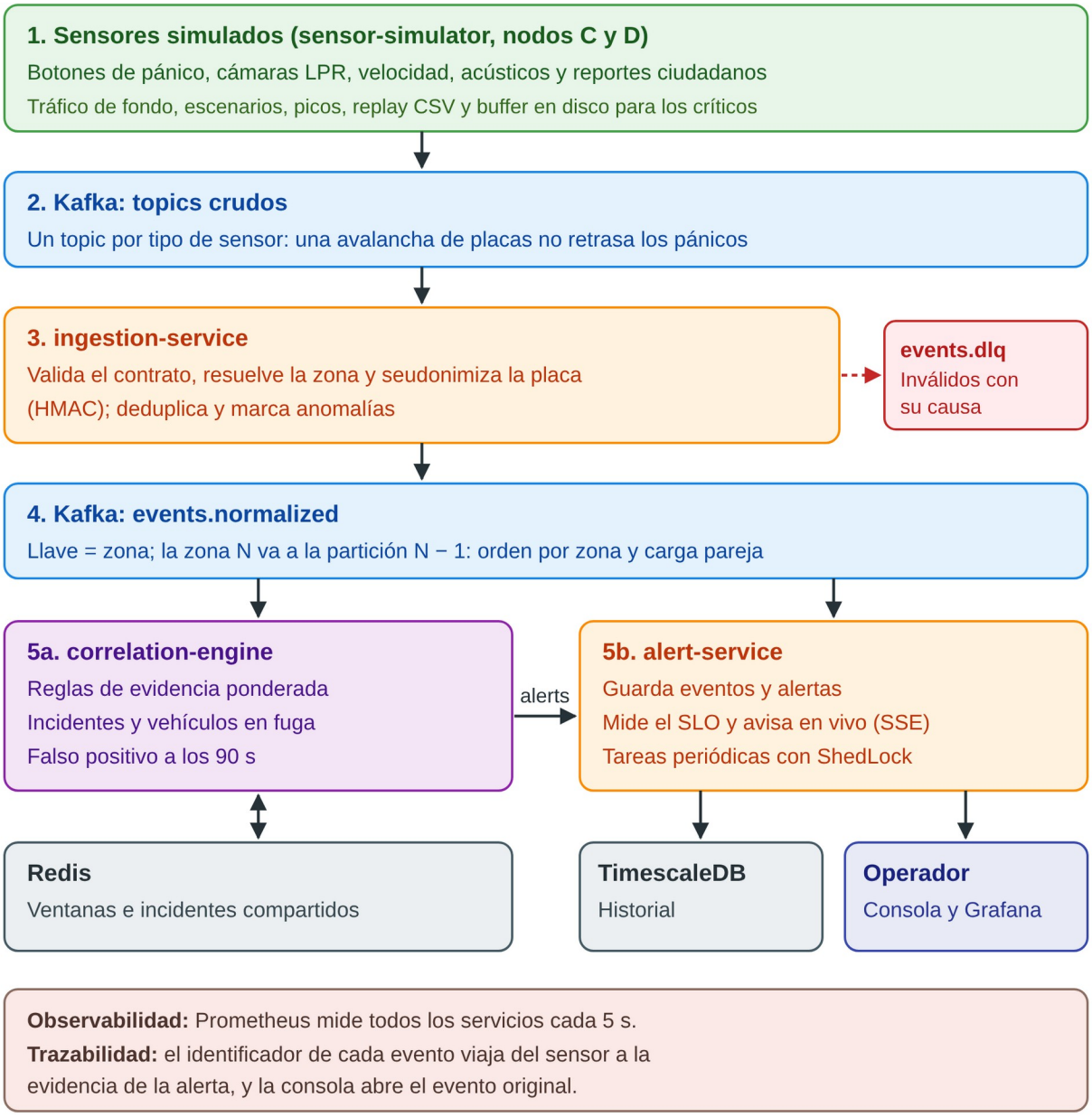
5. Arquitectura propuesta

5.1 Visión general y flujo de eventos

Centinela Urbano se compone de cuatro microservicios desarrollados en Java 21 con Spring Boot 3.5 que se comunican exclusivamente a través de Apache Kafka (ADR-010). La Figura 1 muestra el recorrido de un evento desde el sensor simulado hasta la alerta del operador.

Figura 1

Flujo de eventos del sistema Centinela Urbano



Nota. Elaboración propia.

El flujo tiene cinco etapas. Primero, los sensores simulados publican eventos crudos en un *topic* por tipo de sensor, lo que aísla la carga: una avalancha de lecturas de placas no retrasa a los botones de pánico. Segundo, el servicio de ingesta valida cada evento contra su contrato, resuelve la zona a partir del nombre o de las coordenadas, lo

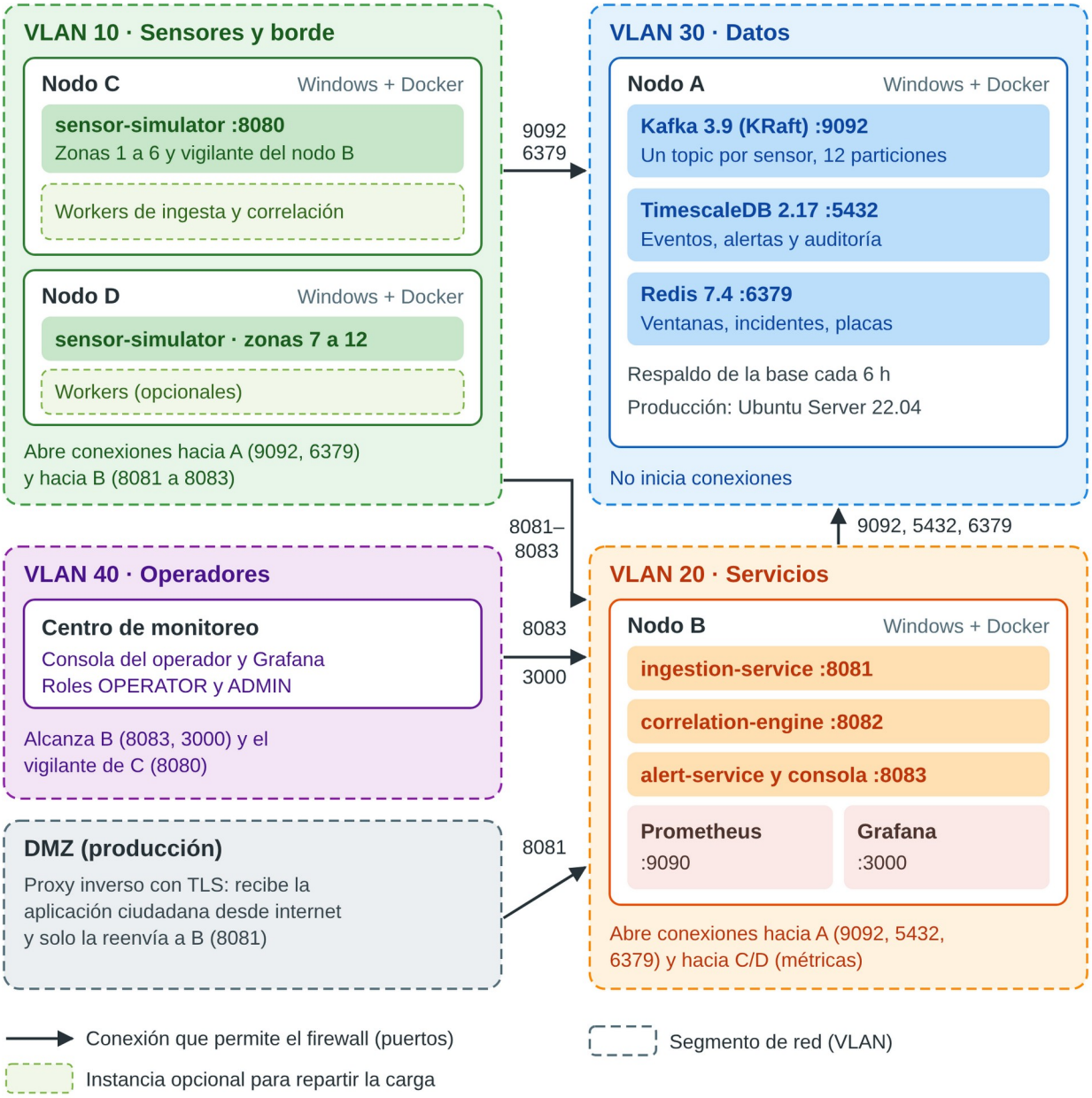
convierte a un formato común, reemplaza la placa por un seudónimo, deduplica y envía los mensajes inválidos a una cola de mensajes muertos. Tercero, los eventos normalizados se publican con la zona como llave y un particionador propio envía cada zona a su propia partición (la zona N a la partición N − 1), de modo que todos los eventos de una zona llegan en orden a la misma instancia y las doce particiones reciben carga; con el reparto por defecto de Kafka, las doce llaves caían en solo siete particiones. Cuarto, dos grupos de consumidores independientes leen esos eventos: el motor de correlación, que infiere incidentes en tiempo real, y el almacenamiento histórico en TimescaleDB. Quinto, las alertas se persisten, se miden contra el SLO y se envían en vivo a la consola del operador.

5.2 Infraestructura, nodos y segmentos de red

La Figura 2 presenta el esquemático de infraestructura on-premise, que sigue la distribución por nodos sugerida en el enunciado y la complementa con segmentación de red, y la Tabla 2 detalla los componentes de cada nodo.

Figura 2

Esquema de infraestructura: nodos, servicios y segmentos de red



Nota. Las instancias con borde punteado son opcionales y permiten repartir la carga entre máquinas. Elaboración propia.

Tabla 2

Distribución de componentes por nodo

Nodo	Función	Componentes	Sistema operativo
A	Datos	Kafka 3.9 (KRaft), TimescaleDB 2. 17 (PostgreSQL 16), Redis 7.4 y respaldo programado de la base	Windows 10/11 con Docker Desktop
B	Microservicios y visualización	ingestion-service, correlation-engine, alert-service, Prometheus, Grafana	Windows 10/11 con Docker Desktop
C / D	Generación de eventos	sensor-simulator (zonas 1-6 y 7-12), vigilante del nodo B e instancias adicionales de ingesta y correlación	Windows 10/11 con Docker Desktop

Nota. Cada nodo tiene su propio archivo Docker Compose en la carpeta deploy del repositorio y se configura y levanta con el script centinela.cmd. El enunciado recomienda Ubuntu Server 22.04 para el nodo A y macOS o Linux para el B; se usó Windows porque es el sistema de las computadoras del laboratorio y, como todo corre en contenedores, los mismos archivos funcionan en producción sobre Ubuntu Server. Elaboración propia.

La red se divide en cuatro VLAN con reglas explícitas de firewall, más una zona desmilitarizada (DMZ) en producción. Los sensores y equipos de borde solo alcanzan Kafka y Redis en el nodo A y los puertos de los servicios del nodo B, cuya salud revisa el vigilante; los servicios alcanzan Kafka, PostgreSQL y Redis y leen las métricas de los generadores; el segmento de datos no inicia conexiones; y los operadores solo alcanzan la consola y Grafana en el nodo B y el vigilante del nodo C, que les informa si cae B. La DMZ recibe la aplicación ciudadana desde internet y solo la reenvía a la puerta de entrada del nodo B. Esta segmentación aplica el principio de mínimo privilegio documentado en el ADR-007.

Como las computadoras del laboratorio usan Windows, todo el sistema se instala y se opera desde Windows con un único script (centinela.cmd) que no requiere más que Docker Desktop: configura cada nodo, abre los puertos en el firewall, detecta los puertos reservados por Hyper-V (el error 10013 de Windows) y elige otros libres, levanta los nodos en orden y permite simular caídas, consultar la cola de errores y respaldar la base. Un requisito del grupo fue poder ejecutar el sistema en más de una máquina a la vez para distribuir la carga. Los *topics* de alto volumen tienen 12 particiones y los servicios de ingesta y correlación se pueden replicar en los nodos C y D; Kafka reparte automáticamente las particiones entre todas las instancias de un mismo grupo y las reasigna si una de ellas falla.

5.3 Componentes y responsabilidades

La Tabla 3 resume la responsabilidad de cada microservicio y la forma en que escala. Los cuatro comparten una librería con los contratos de eventos, el catálogo de zonas y la seudonimización, de modo que un cambio en un contrato se hace una sola vez y el compilador señala cada lugar afectado.

Tabla 3

Microservicios y sus responsabilidades

Componente	Responsabilidad principal	Escalamiento
sensor-simulator	Emular los cinco sensores, escenarios, picos de carga, reproducción CSV, buffer en disco con escritura anticipada, validación automática de correlaciones y vigilancia del nodo B	Una instancia por nodo generador, cada una con un subconjunto de zonas
ingestion-service	Validación, normalización, seudonimización, deduplicación, cola de	Instancias del grupo de consumidores de ingesta

Componente	Responsabilidad principal	Escalamiento
	mensajes muertos y puerta de entrada REST con buffer en disco para la aplicación móvil	
correlation-engine	Reglas de evidencia ponderada, incidentes, falsos positivos, seguimiento de vehículos y abuso de dispositivos	Instancias del grupo de correlación, con estado compartido en Redis
alert-service	Persistencia, API REST, consola del operador en vivo, seguridad por roles y tareas periódicas, incluido el reproceso de la cola de mensajes muertos	Varias instancias; las tareas se coordinan con ShedLock

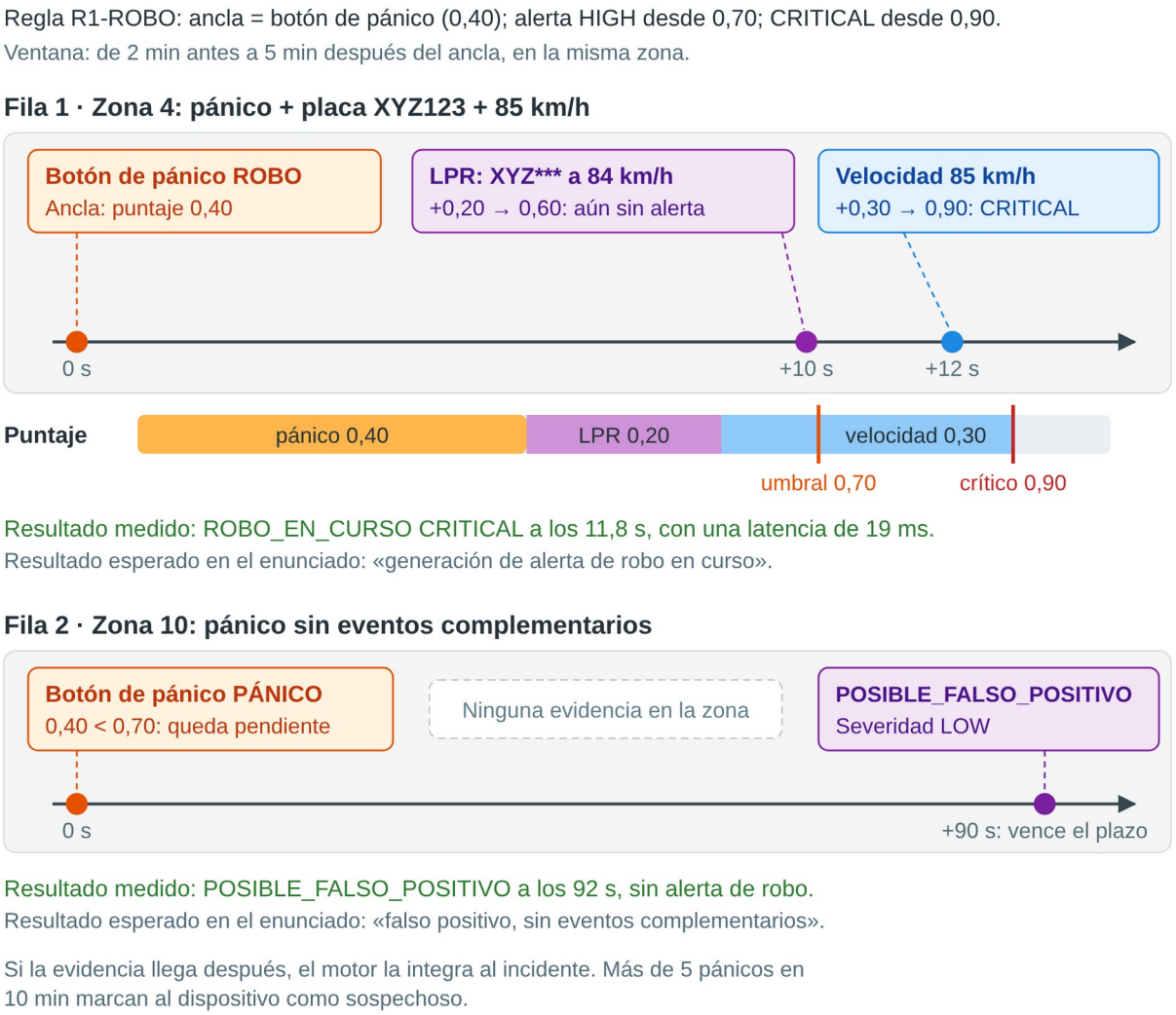
Nota. Todos los servicios exponen métricas para Prometheus y verificación de salud. Elaboración propia.

5.4 Modelo de correlación e inferencia

El motor de correlación implementa reglas de **evidencia ponderada** configurables en un archivo YAML (ADR-009). Cada regla define un evento ancla, que inicia la sospecha, y un conjunto de evidencias con peso. El puntaje de un incidente es la suma del peso del ancla y de las evidencias encontradas en la misma zona dentro de la ventana de tiempo, con un máximo de 1,0. Si el puntaje alcanza el umbral (0,70) se genera la alerta; si alcanza el umbral crítico (0,90), la severidad sube a crítica. Una misma persona o sensor no puede corroborarse a sí mismo. La Figura 3 ilustra cómo el motor resuelve los dos casos de la tabla 2.4 del enunciado.

Figura 3

Resolución de la tabla 2.4 mediante evidencia ponderada en una ventana de tiempo



Nota. Los tiempos y latencias corresponden a una ejecución real del escenario tabla-2-4 del simulador. Elaboración propia.

La Tabla 4 resume las siete reglas configuradas, con el peso de cada ancla y de sus evidencias principales.

Tabla 4

Reglas de correlación configuradas

Regla	Ancla (peso)	Evidencias principales	Resultado
R1 Robo en curso	Botón de pánico de robo, asalto o pánico (0,40)	Exceso de velocidad (0,30), vehículo veloz en LPR (0,20), placa robada (0,20), disparo o vidrios rotos (0,15), otro testigo (0,10)	ROBO_EN_CURSO
R2 y R3 Accidente vial	Reporte de accidente (0,45) o choque detectado (0,50)	Choque o vidrios rotos, exceso de velocidad, otros reportes	ACCIDENTE_VIAL
R4 Disparos	Disparo con probabilidad de al menos 0,7 (0,50)	Disparo en otro sensor, botón de pánico, reporte de altercado	DISPAROS_CONFIRMADOS
R5 Incendio	Reporte o pánico de incendio (0,45)	Otro reporte de otra fuente, explosión detectada	INCENDIO_ESTRUCTURAL
R6 Vehículo robado	Placa en la lista de robados (0,80)	Exceso de velocidad (sensor de velocidad o la misma placa)	VEHICULO_REPORTADO
R7 Emergencia médica	Botón de emergencia médica (1,0)	No requiere corroboración	EMERGENCIA_MEDICA

Nota. Además, el motor detecta dispositivos que abusan del botón de pánico, sigue entre zonas la placa de un incidente (vehículo en fuga) y clasifica como posible falso positivo toda ancla no corroborada en 90 segundos. Elaboración propia.

Para mitigar falsos positivos, el motor exige corroboración antes de alertar un robo, exige que un disparo lo confirme otra clase de fuente, un botón de pánico o un

reporte, lo que filtra la pirotecnia, marca como sospechoso a un dispositivo que presiona el botón más de cinco veces en diez minutos y agrupa en un solo incidente las correlaciones del mismo tipo en la misma zona, actualizándolo cuando se suma un nuevo testigo o cuando sube la severidad. Cada alerta incluye su explicación: el puntaje, el umbral y la lista de evidencias con su peso y un detalle legible, como la velocidad medida, la placa enmascarada o el texto del reporte ciudadano, y cada evidencia enlaza con el evento original guardado, lo que permite justificar la inferencia ante un operador, un juez o la ciudadanía. Finalmente, una tarea horaria calcula la precisión de cada regla por zona a partir de las validaciones de los operadores y ajusta los umbrales: si en una zona la precisión baja del 50 %, el motor exige más evidencia.

Un caso delicado es la recuperación tras una falla. Si Redis o Kafka se detienen, los eventos esperan en Kafka y, al volver, el motor recibe de golpe un pánico junto con la evidencia que lo corrobora. Para no declarar como falso positivo un ancla cuya evidencia todavía está en la cola, el motor detecta que se está poniendo al día sin depender de los relojes de las máquinas, que pueden estar desfasados: por el atraso que informa el propio consumidor de Kafka (más de 50 mensajes en una partición), por los eventos reenviados desde un buffer en disco y por las particiones que no logran procesar. Mientras dura esa condición, y hasta 40 segundos después, la clasificación de falsos positivos espera; lo mismo ocurre al arrancar y cada vez que Kafka reasigna particiones.

5.5 Visualización y consola del operador

La visualización se resuelve con tres tableros de Grafana provisionados como código: el centro de monitoreo en tiempo real (Figura 4), los SLO y la salud del sistema (Figura 5) y el análisis de falsos positivos (Figura 6). Grafana consulta TimescaleDB con un usuario de solo lectura que únicamente accede a vistas sin seudónimos, y Prometheus para las métricas técnicas. La consola del operador (Figura 7) muestra las alertas en vivo mediante *Server-Sent Events*, con su explicación y el detalle de cada evidencia, abre el evento original que respalda cada una y permite confirmar un incidente o marcarlo como falso positivo, información que alimenta el ajuste adaptativo de umbrales. Las alertas del propio sistema, como un incumplimiento de SLO, llegan por el mismo canal.

Figura 4

Tablero de Grafana del centro de monitoreo en tiempo real



Nota. Mapa de las doce zonas con el calor de los eventos de riesgo y las alertas por severidad, alertas recientes con las críticas primero y eventos y alertas por minuto, durante la validación final del 25 de octubre de 2026. Captura propia.

Figura 5

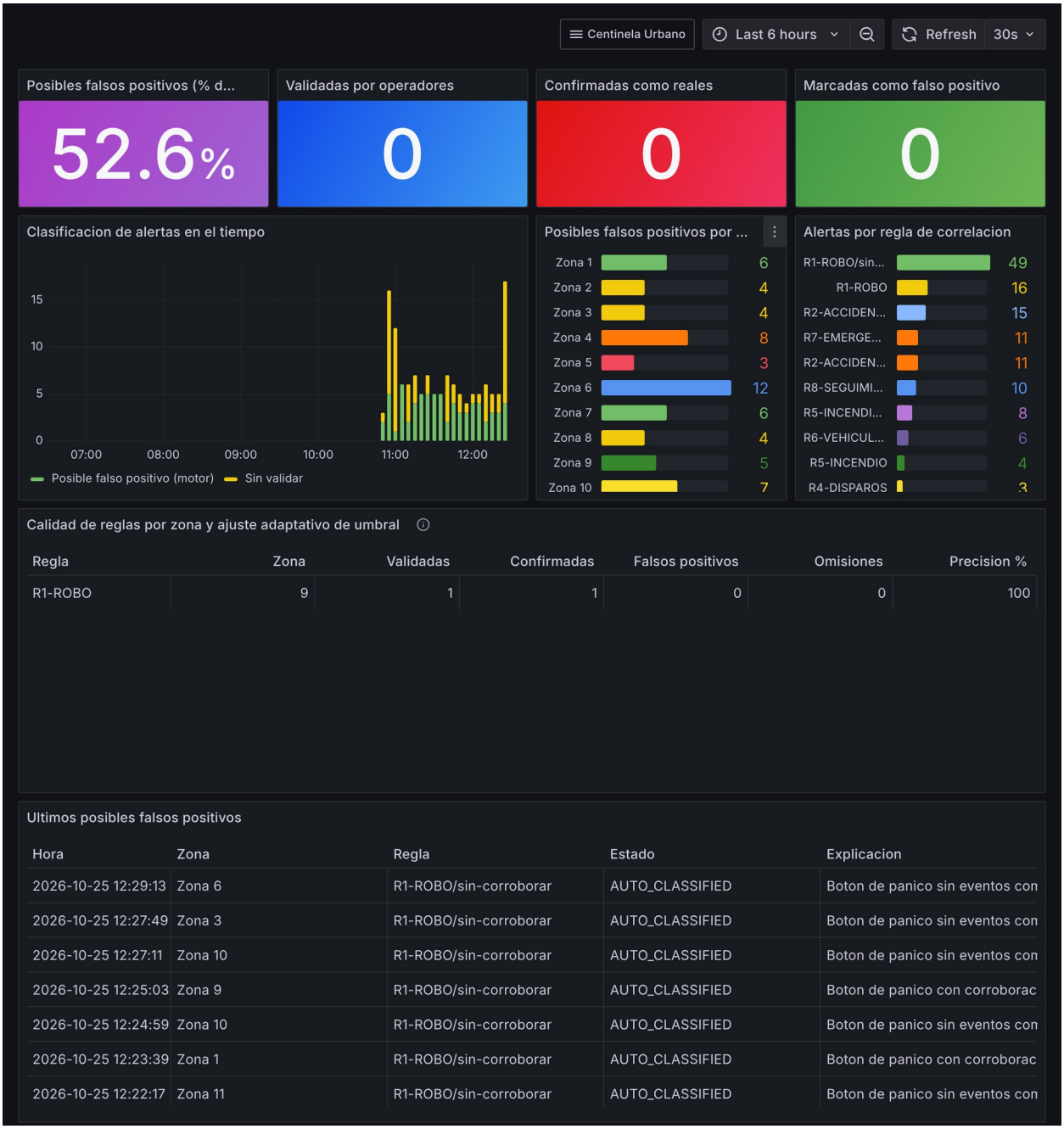
Tablero de Grafana de SLO y salud del sistema



Nota. Cumplimiento del SLO de latencia, percentiles, frescura de datos, eventos críticos perdidos (SLO-2) y rezago (SLO-4), tasa de ingesta, rezago de los consumidores, latencia por etapa y memoria de cada servicio. Captura propia.

Figura 6

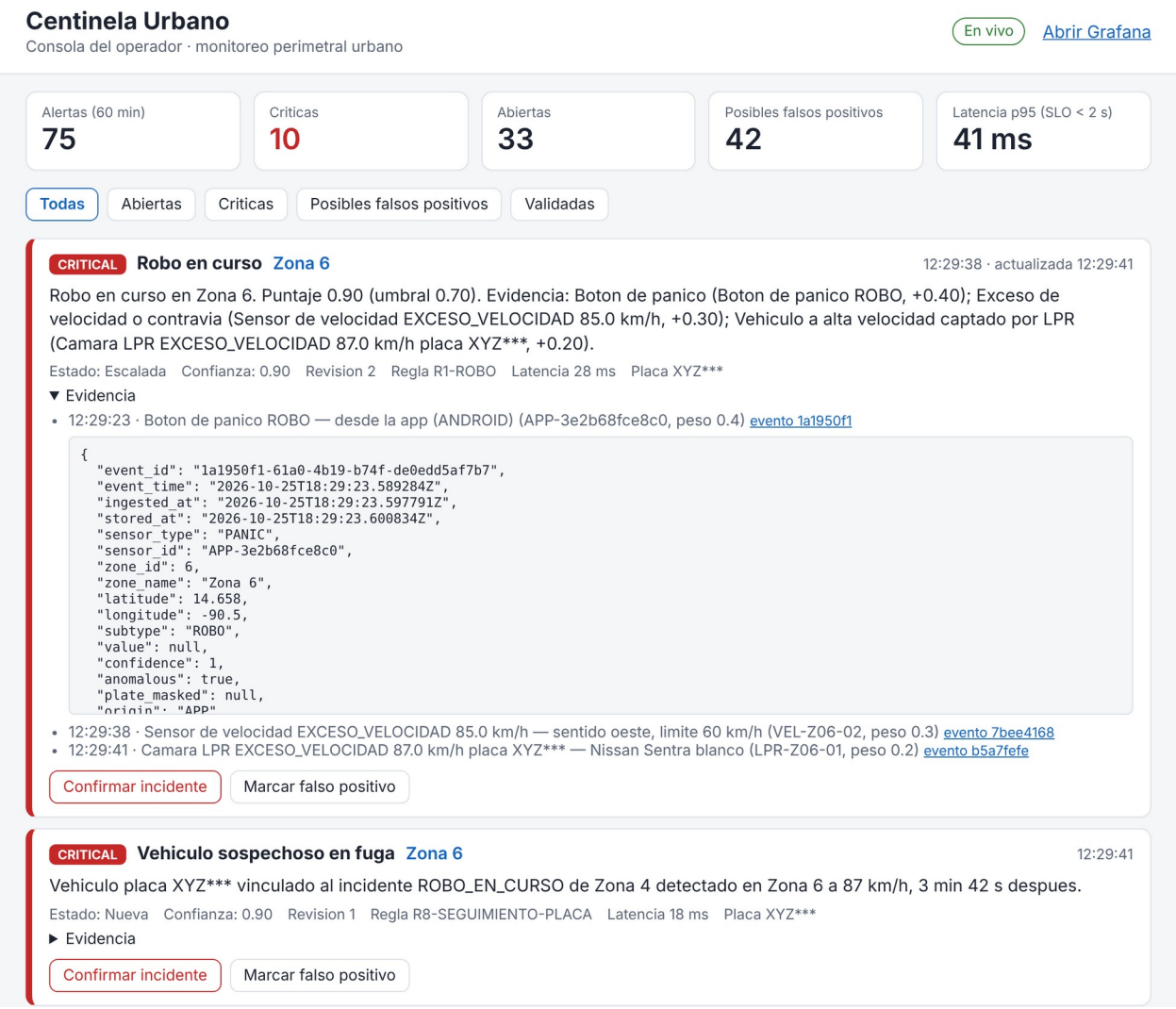
Tablero de Grafana de análisis de falsos positivos



Nota. Proporción de posibles falsos positivos, validaciones de los operadores, clasificación de alertas en el tiempo, posibles falsos positivos por zona y por regla, calidad de reglas con su ajuste de umbral y últimos casos clasificados. Captura propia.

Figura 7

Consola del operador con alertas explicadas en vivo



Nota. Alerta de robo con su evidencia abierta y el evento original de la primera evidencia, recuperado de la base de datos. Debajo, el vehículo en fuga vinculado por su placa a un incidente anterior. Captura propia.

6. Justificación técnica de las decisiones

Las decisiones de arquitectura se documentaron en diez ADRs con el formato obligatorio de la sección 4.1: contexto, opciones consideradas con sus *trade-offs*, decisión, consecuencias e impacto en el negocio. La Tabla 5 resume cada decisión; el detalle completo se encuentra en el Anexo A.

Tabla 5

Resumen de las decisiones de arquitectura

ADR	Decisión	Alternativas evaluadas	Razón principal
001	Despliegue on-premise en la LAN municipal	Híbrido, cloud-native	Control de datos personales, funcionamiento sin internet y costo predecible
002	Apache Kafka en modo KRaft	RabbitMQ, servicio gestionado	Varios consumidores independientes, orden por zona y reparto de carga
003	TimescaleDB como única base de datos	PostgreSQL puro, InfluxDB, NoSQL y Elasticsearch	Series de tiempo y datos relacionales con SQL en una sola base
004	Spring Scheduler con ShedLock	Apache Airflow, Quartz en clúster	Siete tareas simples no justifican la complejidad de Airflow
005	Docker Compose por nodo	Instalación directa, Kubernetes	Simplicidad y portabilidad para tres o cuatro máquinas
006	DLQ con reproceso, reintentos según la clase de falla, escritura anticipada en disco y vigilante del nodo B	Número fijo de reintentos, buffer solo tras fallar el envío	Cero pérdida de eventos críticos y caídas visibles
007	Seudonimización HMAC, retención de 30 días y	Placa en claro, cifrado reversible, hash simple	Una fuga no revela placas ni identidades

ADR	Decisión	Alternativas evaluadas	Razón principal
	mínimo privilegio		
008	Cuatro SLO medibles con degradación controlada	Solo tableros, observabilidad completa	El sistema actúa ante un incumplimiento
009	Motor de evidencia ponderada con ventanas en Redis	Reglas en código, Kafka Streams, Flink, aprendizaje automático	Explicabilidad y ajuste sin recompilar
010	Microservicios orientados a eventos en Java y Spring Boot	Monolito modular, microservicio por sensor; Python o Go como lenguaje	Reparto de carga real; un solo lenguaje con el cliente oficial de Kafka

Nota. Elaboración propia con base en los ADR del Anexo A.

6.1 Plataforma de despliegue (ADR-001)

Se eligió un despliegue on-premise porque responde a las tres prioridades del cliente. Los datos de placas y ubicaciones nunca salen de la red municipal; la detección funciona aunque falle el enlace a internet, cumpliendo la consideración del enunciado de no depender de latencias externas no controladas para la ingesta; y el costo es predecible. La Tabla 6 compara el costo estimado a tres años de las tres alternativas.

Tabla 6

Estimación de costo de orden de magnitud a tres años (USD)

Concepto	On-premise	Híbrido	Cloud-native
Inversión inicial	≈ 12 000	≈ 7 000	≈ 0
Costo anual	≈ 7 500	≈ 9 700-11 000	≈ 16 000-19 000 (incluye enlace dedicado)
Total a tres años	≈ 34 500	≈ 36 000-40 000	≈ 48 000-57 000
Funciona sin	Sí	Parcialmente	No

Concepto	On-premise	Híbrido	Cloud-native
internet			

Nota. Estimación con precios de lista aproximados de 2026, sujeta a cotización. Los precios de la nube corresponden a los servicios gestionados de Amazon Web Services en la región us-east-1 (Amazon Web Services, s. f.-a, s. f.-b, s. f.-c). Elaboración propia con base en el ADR-001.

La latencia extremo a extremo esperada, sumando la red local, la ingesta, la consulta en memoria a Redis y la persistencia, era de 20 a 105 milisegundos. La medición lo confirmó: medianas de 44 a 78 milisegundos en operación normal y de 39 milisegundos en el pico de diseño. La desventaja aceptada es la falta de elasticidad automática, que se mitiga dimensionando para un pico de veinte veces el tráfico normal, con degradación controlada y con escalamiento horizontal en los nodos C y D.

6.2 Broker y persistencia (ADR-002 y ADR-003)

Kafka se prefirió sobre RabbitMQ porque tres procesos distintos deben leer los mismos eventos a su propio ritmo (correlación, almacenamiento e ingesta) y porque su modelo de registro persistente permite reprocesar y recuperarse tras una caída sin pérdida. La partición por zona garantiza el orden necesario para correlacionar y habilita el reparto de carga entre máquinas; el particionador propio asigna a cada zona su propia partición para que la carga se reparta de manera pareja. Para la persistencia, TimescaleDB permitió usar una sola base de datos para series de tiempo (eventos) y datos relacionales (alertas, auditoría y lista de placas), con compresión, retención automática y agregados continuos. Se descartó Elasticsearch del stack de referencia porque la búsqueda histórica requerida se resuelve con índices de TimescaleDB y su incorporación duplicaría datos y consumiría alrededor de 2 GB adicionales de memoria.

6.3 Tareas periódicas y contenedores (ADR-004 y ADR-005)

El enunciado pregunta si se justifica la complejidad operativa de Airflow. Las tareas del sistema son siete, cortas e independientes: monitor de SLO, retención de

placas, expiración de alertas, calidad de reglas, sincronización de la lista de placas robadas, su revisión cada 15 segundos y el reproceso de la cola de mensajes muertos. Airflow agregaría entre cuatro y cinco contenedores, de 2 a 4 GB de memoria y un segundo lenguaje, porque sus flujos se escriben en Python, en un sistema que el equipo decidió escribir por completo en Java (sección 6.5). Se optó por Spring Scheduler con ShedLock, que garantiza que cada tarea se ejecute en una sola instancia aunque existan varias. De forma análoga, se decidió conscientemente no usar Kubernetes: con tres o cuatro máquinas su plano de control consumiría una parte significativa de los recursos y su red superpuesta añadiría puntos de falla el día de la demostración, mientras que el escalamiento que el sistema necesita ya lo resuelven los grupos de consumidores de Kafka.

6.4 Resiliencia, seguridad y observabilidad (ADR-006 a ADR-008)

La política de resiliencia parte de reconocer que no todos los eventos valen lo mismo. Los botones de pánico, los reportes ciudadanos y los disparos tienen pérdida aceptable cero: se escriben en un diario en disco antes de enviarse (escritura anticipada) y se reenvían cuando Kafka vuelve, tanto en los nodos generadores como en la pasarela de la aplicación móvil. La telemetría que puede corroborar un incidente (placas, excesos de velocidad y sonidos relevantes) se conserva en disco durante 15 minutos como máximo, mientras que las lecturas normales pueden perderse durante una caída del broker. Cada falla se clasifica: los mensajes inválidos van a una cola de mensajes muertos sin reintentos, los errores transitorios (Redis, la base o Kafka no disponibles) se reintentan sin límite con espera exponencial y una tarea periódica reprocesa la cola de mensajes muertos. La entrega es de al menos una vez, con deduplicación por identificador de evento e identificadores deterministas para las alertas, de modo que una alerta reprocesada no se duplica.

La arquitectura separa el estado (nodo A) del cómputo (nodo B). Si cae el nodo B, Kafka retiene los eventos y las instancias de los nodos C y D siguen detectando; un vigilante que corre en el nodo C anuncia la caída en segundos y la consola del operador muestra un aviso de desconexión y, al reconectar, destaca las alertas que no se vieron a

tiempo. Si cae el nodo A, ningún evento crítico se pierde y, al volver, la lista de placas robadas se reconstruye sola desde la base.

En seguridad, la placa se reemplaza en la puerta de entrada por un seudónimo HMAC-SHA256 con llave secreta. Se descartó el hash simple porque las placas guatemaltecas tienen pocas combinaciones y podrían revertirse por fuerza bruta. El seudónimo permite seguir vehículos y compararlos con la lista de robados sin conocer la placa, se borra a los 30 días y nunca es visible para Grafana, cuyas vistas tampoco muestran el identificador de los celulares de la aplicación. Las placas que un ciudadano escribe en el texto libre de un reporte se enmascaran al entrar, las ubicaciones de ciudadanos se generalizan a unos 110 metros y todas las acciones humanas quedan en una bitácora de auditoría.

En observabilidad se definieron cuatro SLO: latencia de alerta (95 % por debajo de 2 segundos), pérdida de eventos críticos (cero), frescura de datos (por debajo de 10 segundos) y rezago de procesamiento. Cuando el SLO de latencia o de frescura se incumple, el sistema determina primero la causa a partir del rezago de los consumidores de Kafka. Si el rezago es grande y sostenido, se trata de una sobrecarga y el sistema activa automáticamente una degradación controlada: conserva solo una muestra de las lecturas de velocidad normales, el mayor volumen y que no participa en la correlación, y publica una alerta de sistema en la consola del operador. Si el rezago es bajo o va bajando, las alertas tardías corresponden a la puesta al día tras una caída ya superada: se informa al operador, pero no se degrada, porque se descartarían datos justo durante la recuperación. Los otros dos SLO también tienen una acción automática: si un generador no logra guardar un evento crítico, lo informa y el sistema publica una alerta crítica de pérdida, y si la correlación acumula un atraso sostenido, se publica una alerta de rezago. Todas las alertas de sistema llegan en vivo a la consola del operador.

6.5 Estilo arquitectónico y lenguaje (ADR-010)

El enunciado no fija el lenguaje de programación y sugiere scripts de Python para simular eventos. Escribir todo el sistema en Java 21 con Spring Boot fue una preferencia del equipo, y el ADR-010 la sostiene con razones técnicas frente a Python y Go. Primero, Kafka se desarrolla en Java y Scala, y su cliente oficial es Java: el particionador por zona

es una clase Java que el cliente carga directamente, y Spring for Apache Kafka aporta los reintentos y el manejo de errores que exige la política de resiliencia. Segundo, un solo lenguaje permite que el simulador y los servicios compartan los mismos contratos de eventos con tipado estático: un campo nuevo se cambia una vez y el compilador señala cada lugar afectado, mientras que con un simulador en Python habría que mantenerlos en dos lenguajes. Tercero, Spring Boot ya integra lo que el sistema necesita, como la verificación de salud, las métricas para Prometheus, la seguridad por roles, las migraciones de la base de datos y las tareas con bloqueo distribuido, sin piezas adicionales que elegir y unir.

El costo aceptado es el consumo de recursos: cada servicio usa entre 280 y 390 MB de memoria y tarda de 10 a 16 segundos en arrancar, más que un equivalente en Go o Python. Ese costo no compromete los objetivos del sistema: en el pico de diseño el percentil 95 de la latencia fue de 67 milisegundos, y mientras un servicio arranca Kafka retiene los eventos, de modo que no se pierde ninguno.

7. Evaluación de resultados

7.1 Metodología

Las pruebas se ejecutaron entre el 9 y el 25 de octubre de 2026 en un equipo de cuatro núcleos y 16 GB de memoria, primero con todos los servicios en una sola composición y después en modo multi-nodo, con los nodos A, B y C levantados como proyectos independientes conectados por dirección IP. Como todos los servicios compartían los mismos núcleos, los resultados representan un límite inferior respecto al hardware dedicado propuesto. Se evaluaron seis aspectos: la validación de correlaciones esperadas, los falsos positivos con tráfico de fondo, la integración y trazabilidad de los datos, el comportamiento ante picos de carga, la resiliencia ante fallas y la distribución de carga entre máquinas. Para la resiliencia se apagó de verdad cada servidor con escenarios en curso y la consola del operador abierta en un navegador automatizado. Además, el proyecto incluye 119 pruebas unitarias automatizadas: 28 sobre el motor de correlación, que usan las mismas reglas de configuración, y el resto sobre el diario en disco, la clasificación de fallas, la deduplicación, la seudonimización, el particionador por

zona, la pasarela de la aplicación, el aislamiento de escenarios, el vigilante y el monitor de SLO.

7.2 Validación de correlaciones esperadas

El simulador incluye una validación automática que lanza en paralelo los ocho escenarios de los casos de uso, cada uno en una zona distinta, y verifica en la API de alertas que aparezcan las nueve alertas esperadas con su severidad mínima dentro de su tiempo límite. La Tabla 7 muestra la corrida final en modo multi-nodo.

Tabla 7

Validación automática de correlaciones esperadas (corrida final, multi-nodo)

Escenario	Alerta esperada	Zona	Severidad	Detectada a los	Latencia
Robo en curso (tabla 2.4, fila 1)	ROBO_EN_CURSO	4	Crítica	18,0 s	44 ms
Fuga del vehículo	VEHICULO_EN_FUGA	8	Crítica	90,0 s	28 ms
Falso positivo (tabla 2.4, fila 2)	POSIBLE_FALSO_POSITIVO	10	Baja	91,0 s	No aplica
Accidente vial	ACCIDENTE_VIAL	7	Crítica	20,0 s	78 ms
Disparos	DISPAROS_CONFIRMADOS	2	Crítica	10,1 s	65 ms
Incendio	INCENDIO_ESTRUCTURAL	11	Crítica	15,1 s	74 ms

Escenario	Alerta esperada	Zona	Severidad	Detectada a los	Latencia
Vehículo robado	VEHICULO_REPORTADO	12	Alta	0,3 s	440 ms
Botón abusivo	DISPOSITIVO_SOSPECHOSO	1	Media	25,0 s	42 ms
Emergencia médica	EMERGENCIA_MEDICA	5	Alta	0,3 s	477 ms

Nota. «Detectada a los» indica el tiempo desde el inicio del escenario hasta que la alerta alcanzó la severidad esperada, que depende del guion de cada escenario. La latencia mide desde el evento que completó la correlación hasta la alerta guardada. Resultado: 9 de 9. Elaboración propia.

Los dos casos de la tabla 2.4 se resolvieron como se esperaba. El robo en curso se clasificó como crítico al llegar la lectura de la placa, con una latencia de 44 milisegundos. El botón de pánico sin eventos complementarios se clasificó como posible falso positivo al vencer el plazo de corroboración, sin generar una alerta de robo. En el escenario que reproduce la tabla 2.4 tal cual (Figura 3), el robo pasó a crítico al llegar la lectura de velocidad, a los 11,8 segundos del pánico y con una latencia de 19 milisegundos, y el pánico de la zona 10 quedó como posible falso positivo a los 92 segundos. En la revisión final la validación se repitió en modo multi-nodo con el tráfico de fondo encendido y dio 9 de 9 en todas las corridas.

7.3 Análisis de falsos positivos

El tráfico de fondo no contiene incidentes reales: sus botones de pánico, reportes y sonidos se generan al azar, igual que los excesos de velocidad. Por eso sirve para medir los falsos positivos del propio modelo. Se dejó el sistema una hora en operación normal, sin escenarios y en modo multi-nodo, y se clasificó cada ancla del fondo según lo que hizo el motor con ella. La Tabla 8 resume el resultado.

Tabla 8

Destino de las anclas del tráfico de fondo durante una hora de operación normal

Regla	Anclas	Posible falso positivo	Alerta por coincidencia	Sumada a un incidente abierto
R1 Robo en curso	31	24 (77 %)	6 (19 %)	1
R2 Accidente vial (reporte)	17	9 (53 %)	7 (41 %)	1
R5 Incendio	7	7 (100 %)	0	0
Total de las reglas con verificación	55	40 (73 %)	13 (24 %)	2
R7 Emergencia médica	7	No aplica	7 (no requiere corroboración)	0

Nota. Medición del 25 de octubre de 2026, de 17:20 a 18:20 UTC, con 48 513 eventos de fondo (13,5 por segundo). En esa hora no hubo anclas de las reglas R3 y R4 (choque o disparo con probabilidad de al menos 0,7). Elaboración propia.

El motor clasificó como posible falso positivo el 73 % de las anclas que exigen corroboración, que es la respuesta correcta ante un fondo sin incidentes. El 24 % restante alcanzó el umbral por coincidencia, casi siempre de la misma manera: diez de las trece alertas sumaron exactamente 0,70, nueve de ellas con una sola evidencia, un exceso de velocidad o una contravía al azar en la misma zona, y doce quedaron en severidad alta, sin llegar a crítica. La causa es aritmética: el fondo produjo unas tres anomalías de velocidad por zona y por hora, de modo que una ventana de siete minutos alrededor de un pánico contiene una con una probabilidad cercana al 30 %. El accidente por reporte es la regla más expuesta, porque su ancla pesa 0,45 y le basta cualquier exceso de velocidad (0,25) para llegar al umbral.

El diseño ya contempla este ruido: las validaciones de los operadores alimentan la tarea de calidad de reglas, que sube el umbral 0,10 en la regla y la zona donde la

precisión baja del 50 % con al menos cinco validaciones; con un umbral de 0,80, un ancla necesita al menos dos evidencias, y once de las trece coincidencias medidas no habrían generado alerta. La misma coincidencia explicaba el defecto de la revisión final en el que la fila del falso positivo de la tabla 2.4 terminaba como robo con el fondo encendido, y por eso los escenarios se aíslan del ruido. Las emergencias médicas no se verifican por diseño: quien pide ayuda médica recibe atención sin esperar corroboración.

7.4 Integración, trazabilidad y visualización

El enunciado pide medir la capacidad de integración, trazabilidad, visualización y toma de decisiones con datos no estructurados y heterogéneos. La Tabla 9 resume cómo responde el sistema a cada una y la prueba que lo comprobó.

Tabla 9

Capacidad de integración, trazabilidad y visualización

Capacidad	Cómo se resuelve	Comprobación
Integración de fuentes heterogéneas	Cinco contratos distintos se convierten a un evento normalizado común; la zona se resuelve por nombre («Zona 4», «Z4», «4») o por coordenadas	Los cinco tipos de sensores alimentan las mismas reglas; un JSON roto, una hora en el futuro y un tipo de alerta inexistente fueron a la cola de mensajes muertos con su causa
Datos no estructurados	El texto libre de los reportes se conserva, recortado a 200 caracteres y con las placas enmascaradas, y se muestra como detalle de la evidencia	Un reporte enviado con la placa P-512KLM en el texto apareció en la consola como «P51***»
Trazabilidad	El identificador de cada evento viaja desde el sensor hasta la evidencia de la	Las 12 evidencias de las seis alertas correlacionadas más recientes abrieron su evento

Capacidad	Cómo se resuelve	Comprobación
	alerta; la consola abre el evento guardado y la bitácora registra cada acción humana	original
Visualización y toma de decisiones	Consola con alertas explicadas en vivo y validación con un clic; Grafana con mapa de las zonas, tendencias, SLO y falsos positivos	Las validaciones de los operadores alimentan el ajuste de umbrales por zona

Nota. Elaboración propia.

7.5 Comportamiento ante picos de carga

La Tabla 10 compara la latencia de las alertas bajo operación normal, en el pico de diseño y en una sobrecarga de cien veces el tráfico normal.

Tabla 10

Latencia de alertas bajo distintos niveles de carga

Prueba	Eventos por segundo	Mediana	Percentil 95	Máximo	Cumple SLO
Operación normal	13,5	44-78 ms	menos de 500 ms	477 ms	Sí
Pico ×20, primera versión, un nodo	270	253 ms	608 ms	831 ms	Sí
Pico ×20, versión final, multi-nodo	270	39 ms	67 ms	80 ms	Sí
Sobrecarga ×100,	≈ 820 de 1	37 s	240 s	275 s	No

Prueba	Eventos por segundo	Mediana	Percentil 95	Máximo	Cumple SLO
primera versión	350				
Sobrecarga ×100, motor optimizado	≈ 850 de 1 350	70 ms	5,0 s	8,4 s	No

Nota. En ambas sobrecargas el sistema activó automáticamente la degradación controlada y publicó la alerta de sistema SLO_INCUMPLIDO. Elaboración propia.

En el pico de diseño, equivalente a un concierto o un feriado, la versión final cumplió el objetivo de 2 segundos con un margen de unas treinta veces. La prueba de sobrecarga se utilizó para encontrar el punto de quiebre: el rezago de los consumidores medido con Prometheus identificó al motor de correlación como cuello de botella, con 5 762 mensajes pendientes. Se aplicaron dos optimizaciones propias del procesamiento de eventos complejos: el filtrado temprano de los eventos que no cumplen ninguna condición de ninguna regla (alrededor del 99 % de la telemetría normal) y el envío en un solo viaje de red de los comandos de cada ventana a Redis. Con ellas el rezago del motor quedó en cero y la mediana pasó de 37 segundos a 70 milisegundos. Más allá de unas cinco veces el pico de diseño, la respuesta prevista es el escalamiento horizontal.

7.6 Resiliencia y distribución de carga

La Tabla 11 resume las pruebas de resiliencia. Las caídas de nodos se provocaron apagando de verdad cada servidor, y las tres últimas filas corresponden a la revisión final: dos congelamientos de Redis, para comprobar la detección de la puesta al día, y un arranque simultáneo de todos los nodos.

Tabla 11

Resultados de las pruebas de resiliencia

Prueba	Procedimiento	Resultado
Caída del nodo B (93 s)	Se apagó el servidor de servicios y se lanzó el escenario	El vigilante del nodo C anunció la caída a los 16 s;

Prueba	Procedimiento	Resultado
	de disparos	la instancia del nodo C generó la alerta de disparos 10 s después del escenario; la consola avisó la desconexión y al reconectar destacó la alerta que llegó 91 s tarde; sin pérdidas y sin activar la degradación
Caída del nodo A (82 s)	Se apagaron Kafka, la base y Redis durante un robo en curso y un pánico enviado por la aplicación	La pasarela respondió «en cola»; el simulador guardó 4 eventos críticos y 222 de telemetría útil; al volver, el robo se clasificó como robo en curso crítico, el pánico generó su alerta y la lista de placas robadas se recargó sola en 20 s; cero eventos críticos perdidos
Caída de Redis (45 s)	Se detuvo solo Redis durante el escenario de disparos	Ningún mensaje fue a la cola de mensajes muertos; la alerta se generó al volver Redis
Terminación abrupta del simulador	Se detuvo Kafka y se terminó el proceso del simulador con kill -9	Al reiniciar, el diario en disco recuperó el evento pendiente y se generó su alerta
Mensajes inválidos	Inyección de tres mensajes con la herramienta de línea de comandos de Kafka	Los tres llegaron a la cola de mensajes muertos clasificados como

Prueba	Procedimiento	Resultado
		permanentes, sin reintentos y sin afectar el resto del flujo
Caída de una instancia	Se apagó la instancia de correlación del nodo C	Kafka reasignó sus particiones al nodo B; una alerta de disparos se generó en 188 ms con el nodo caído
Redis congelado (2,5 min)	Se congeló Redis con el tráfico de fondo encendido	El atraso de la correlación llegó a 1 631 mensajes; a los dos minutos el sistema publicó la alerta de rezago sostenido, que llegó en vivo a la consola, y al volver Redis informó la puesta al día sin activar la degradación ni declarar falsos positivos por error
Redis congelado (2 min) durante un robo	Se congeló Redis a los 5 s de lanzar el escenario de robo; la velocidad y la placa llegaron durante la falla	Al volver Redis, el motor procesó la evidencia retenida en Kafka y clasificó el robo como crítico (134 s de latencia, por la falla), sin declararlo falso positivo; ningún mensaje fue a la cola de mensajes muertos
Arranque simultáneo de todos los nodos	Se reinició el equipo y los doce contenedores arrancaron a la vez, sin orden, como tras un apagón	En unos dos minutos todos estaban sanos; Redis arrancó vacío y la lista de placas robadas se recargó

Prueba	Procedimiento	Resultado
		sola; la validación dio 9 de 9

Nota. Las caídas de nodos se provocaron con el comando caida del script centinela.cmd, con tráfico de fondo y la consola del operador abierta en un navegador automatizado. Elaboración propia.

Con las instancias de los nodos B y C activas, Kafka asignó seis particiones a cada una, y la validación completa dio 9 de 9 también en este modo. Con el particionador por zona, las doce particiones recibieron tráfico, de 36 a 57 eventos cada una en 40 segundos, y cada uno de los doce hilos de consumo atendió una partición; con el reparto por defecto, cinco particiones quedaban vacías. Las correlaciones funcionaron aunque la evidencia de un mismo incidente fuera procesada por instancias distintas, como la fuga de la zona 4 hacia la zona 8, gracias a que el estado vive en Redis compartido.

7.7 Hallazgos y mejoras

Probar los escenarios de falla, y no solo el camino feliz, permitió descubrir y corregir defectos reales en dos rondas. En la primera se encontraron cuatro. En la primera prueba de caída de Kafka, el buffer en disco no podía escribirse porque el volumen se creaba con permisos de administrador y la aplicación corre sin privilegios. Al repetir el escenario de robo, el seguimiento de la placa no generaba alertas en zonas ya visitadas por un incidente anterior, porque el registro de zonas era por placa y no por incidente. Un incidente abierto absorbía nueva evidencia sin informarlo al operador, lo que motivó una nueva revisión de tipo «actualizada». Y la corroboración tardía registraba una latencia falsa que activó la degradación sin sobrecarga real. Cada corrección se acompañó de una prueba automatizada o de una nueva ejecución de la validación.

En la segunda ronda se apagó de verdad cada servidor, lo que reveló ocho brechas entre lo que prometía la política de resiliencia y lo que hacía el sistema. Las más visibles fueron que, al volver el nodo A, la lista de placas robadas quedaba vacía hasta diez minutos; que un robo ocurrido durante la caída del nodo A se clasificaba como posible falso positivo, porque solo se guardaba el pánico y no la telemetría que lo corroboraba; que nadie anunciaba la caída del nodo B, ya que la consola era justamente

lo que había caído; y que, al recuperarse, el monitor de SLO confundía la puesta al día con una sobrecarga y activaba la degradación. Las otras cuatro se encontraron leyendo el código: el buffer se escribía solo después de que el envío fallaba, la pasarela de la aplicación móvil no tenía buffer, lo que agotaba los reintentos iba a la cola de mensajes muertos sin que nada lo reprocesara y no había respaldo de la base de datos. Todas se corrigieron con los mecanismos descritos en la sección 6.4 y se volvieron a probar con los resultados de la Tabla 11. Por último, al grabar el video de la presentación con el sistema en vivo aparecieron tres defectos más, que el video muestra fallando y luego corregidos: un segundo robo en una zona con el incidente abierto, entre dos y cinco minutos después del primero, se notificaba como revisión recién a los 90 segundos, porque el motor solo probaba la coincidencia de mayor puntaje (74 s de atraso medido; tras la corrección, 36 ms); la regla de vehículo robado aceptaba como evidencia el exceso de velocidad de una cámara que había leído otra placa; y el mapa de Grafana dependía de internet para el fondo, por lo que se agregó una capa local con las doce zonas. Cada corrección quedó cubierta por una prueba automatizada que falla con el código anterior.

Finalmente, una revisión completa del proyecto contra el enunciado, antes de la entrega, encontró diez defectos más, todos corregidos y probados en vivo en modo multi-nodo. Las doce zonas usaban solo siete de las doce particiones. Con relojes desfasados entre computadoras, el motor se creía atrasado y dejaba de clasificar falsos positivos, por lo que la puesta al día ahora se detecta sin relojes. Con el tráfico de fondo encendido, un exceso de velocidad al azar convertía en robo la fila del falso positivo en cerca del 15 % de las corridas, lo que llevó a aislar cada escenario del ruido de fondo. Los SLO de pérdida y de rezago no tenían una acción y las alertas de sistema no llegaban en vivo a la consola. El texto libre de los reportes no se mostraba al operador. Las reglas que se anunciaban ajustables sin recompilar estaban dentro de la imagen. El identificador de los celulares era visible en Grafana y las placas escritas por un ciudadano quedaban en claro. La reproducción de archivos aceptaba rutas arbitrarias. Por último, la consola tenía tres errores de interacción y el script de Windows, once.

8. Comparación con casos reales de ciudades inteligentes

El diseño de Centinela Urbano dialoga con experiencias reales de monitoreo urbano, cuyas lecciones respaldan varias de sus decisiones. La Tabla 12 resume la comparación.

Tabla 12

Comparación con casos reales de ciudades inteligentes

Caso	Descripción	Lección aplicada en Centinela Urbano
Centro de Operações Rio (Río de Janeiro)	Centro inaugurado en diciembre de 2010 que integra el monitoreo de unas 30 agencias municipales y estatales, impulsado tras inundaciones que causaron casi 70 muertes	Valor de un centro de monitoreo que unifica fuentes; en este proyecto a escala municipal y con integración por eventos
C5 de la Ciudad de México	Más de 14 500 postes con videovigilancia y unos 10 000 botones de auxilio; en 2025 se reportaron 922 cámaras y 123 botones fuera de servicio	El botón de pánico es un sensor urbano válido, pero exige detectar dispositivos defectuosos o con uso indebido
ShotSpotter en Chicago	Solo el 9,1 % de las alertas acústicas que motivaron un despacho policial produjo evidencia de un delito con arma de fuego	Un sensor acústico aislado no basta: se exige corroboración y se miden los falsos positivos
Servicio nacional ANPR del Reino Unido	Las lecturas de placas se eliminan a los 12 meses salvo preservación por un proceso	Necesidad de plazos de retención explícitos; se adoptó un plazo menor (30 días) con

Caso	Descripción	Lección aplicada en Centinela Urbano
	legal	seudonimización

Nota. Elaboración propia con base en Centre for Public Impact (s. f.), U.S. Environmental Protection Agency (s. f.), Chilango (s. f.), Expansión Política (2025), City of Chicago Office of Inspector General (2021) y Home Office (s. f.).

El Centro de Operações de Río de Janeiro surgió tras las inundaciones de 2010 y concentra en una sala de control la información de unas treinta agencias (Centre for Public Impact, s. f.; U.S. Environmental Protection Agency, s. f.). Su principal enseñanza es que el valor proviene de integrar fuentes, que es precisamente lo que hace el motor de correlación de este proyecto, aunque a una escala municipal y con integración basada en eventos en lugar de una sala con pantallas por agencia.

El C5 de la Ciudad de México desplegó miles de postes con cámaras y botones de auxilio (Chilango, s. f.), pero reportes posteriores documentaron centenares de cámaras y botones fuera de servicio (Expansión Política, 2025). Esta experiencia confirma que un sistema basado en botones de pánico debe asumir dispositivos defectuosos, vandalizados o usados de forma indebida, lo que en Centinela Urbano se atiende con la detección de dispositivos sospechosos y con la clasificación de botones no corroborados.

El caso de la detección acústica de disparos en Chicago es especialmente relevante para el modelo de correlación. La Oficina del Inspector General de la ciudad encontró que solo el 9,1 % de las alertas que generaron un despacho policial produjo evidencia de un delito relacionado con armas de fuego (City of Chicago Office of Inspector General, 2021). Por ello, en Centinela Urbano un disparo detectado por un único sensor no genera por sí solo una alerta confirmada: aunque un segundo sensor acústico lo registre, ambos suman 0,65 y no alcanzan el umbral de 0,70, por lo que se necesita además un botón de pánico o un reporte ciudadano, y el disparo aislado se clasifica como posible falso positivo.

Finalmente, la práctica británica con lectores automáticos de placas establece que las lecturas almacenadas en el servicio nacional se eliminan a los 12 meses salvo que se preserven para un proceso judicial (Home Office, s. f.). Centinela Urbano adopta el

mismo principio de plazos explícitos, pero con un periodo menor y con seudonimización desde la entrada, en coherencia con los principios de minimización y limitación del plazo de conservación (Parlamento Europeo y Consejo de la Unión Europea, 2016).

9. Conclusiones

1. Una arquitectura orientada a eventos sobre Kafka permitió desacoplar la detección, la correlación y el almacenamiento, de modo que cada parte escala, falla y se recupera de forma independiente. La partición por zona y los grupos de consumidores hicieron posible repartir el procesamiento entre varias máquinas sin cambios en el código, con reasignación automática ante fallas.
2. El modelo de evidencia ponderada resolvió los casos de la tabla 2.4 y los demás casos de uso con 9 de 9 correlaciones esperadas en cada corrida, incluso con el tráfico de fondo encendido, con latencias de decenas de milisegundos, y produce alertas explicables y trazables hasta el evento original, una condición indispensable en un contexto de seguridad ciudadana. Ante un fondo sin incidentes reales, el 73 % de las anclas quedó como posible falso positivo, y las coincidencias restantes quedaron justo en el umbral, donde el ajuste adaptativo puede filtrarlas.
3. El despliegue on-premise es la opción adecuada para el cliente definido: mantiene los datos personales bajo control municipal, funciona sin internet, cumple el SLO de 2 segundos con amplio margen en el pico de diseño y resulta entre un 28 y un 40 % más económico que la nube a tres años.
4. La privacidad desde el diseño es compatible con la correlación: la seudonimización con HMAC permite seguir vehículos y compararlos con listas de robados sin almacenar ninguna placa en claro y sin que Grafana vea ningún seudónimo.
5. Los SLO solo aportan valor cuando el sistema actúa ante su incumplimiento, y los cuatro definidos tienen una acción automática. La degradación controlada protegió los eventos críticos durante la sobrecarga, y la medición del rezago guio una optimización que redujo la latencia mediana de 37 segundos a 70 milisegundos.

6. Las pruebas de falla resultaron tan importantes como las funcionales: revelaron doce defectos y brechas que no aparecían en el camino feliz, la grabación en vivo de la presentación reveló tres más y la revisión final contra el enunciado, otros diez. Separar el estado (nodo A) del cómputo (nodo B) hace que la caída del servidor de servicios solo produzca retraso, mientras que la del servidor de datos, el punto único de falla aceptado para el alcance académico, no provoca la pérdida de ningún evento crítico gracias a la escritura anticipada en disco.

10. Recomendaciones y trabajo futuro

1. Para producción, eliminar el punto único de falla del nodo A con tres brokers de Kafka con factor de replicación tres, una réplica de PostgreSQL y Redis con Sentinel, y agregar cifrado TLS, autenticación SASL con listas de control de acceso e integración con el directorio de usuarios municipal.
2. Resguardar la llave de seudonimización en un gestor de secretos con rotación anual y sincronizar la hora de todos los nodos mediante NTP.
3. Aprovechar las validaciones de los operadores, que constituyen un conjunto de datos etiquetado, para evaluar en una fase posterior modelos de aprendizaje automático que complementen las reglas.
4. Evaluar un modelo híbrido que envíe a la nube únicamente agregados anónimos para analítica regional, sin exponer datos personales.
5. Considerar Kubernetes y Apache Airflow solo si el sistema crece a más de diez máquinas o a flujos de datos con dependencias complejas, según los disparadores definidos en los ADR-004 y ADR-005.

Referencias

Amazon Web Services. (s. f.-a). *Amazon ElastiCache pricing*. https://aws.amazon.com/elasticache/pricing/

Amazon Web Services. (s. f.-b). *Amazon MSK pricing*. https://aws.amazon.com/msk/pricing/

Amazon Web Services. (s. f.-c). *Amazon RDS for PostgreSQL pricing*. https://aws.amazon.com/rds/postgresql/pricing/

Beyer, B., Jones, C., Petoff, J. y Murphy, N. R. (Eds.). (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media.

Centre for Public Impact. (s. f.). *IoE-based Rio operations center* [Estudio de caso]. https://www.centreforpublicimpact.org/case-study/ioe-based-rio-operations-center/

Chilango. (s. f.). *¿Qué tan efectivos son los botones de auxilio chilangos?* https://www.chilango.com/noticias/reportajes/botones-de-auxilio-en-la-cdmx/

City of Chicago Office of Inspector General. (2021). *The Chicago Police Department's use of ShotSpotter technology*. https://igchicago.org/?p=28541

Congreso de la República de Guatemala. (2008). *Decreto número 57-2008: Ley de Acceso a la Información Pública*. https://www.oas.org/juridico/pdfs/mesicic4_gtm_acceso.pdf

Endsley, M. R. (1995). Toward a theory of situation awareness in dynamic systems. *Human Factors*, *37*(1), 32–64. https://doi.org/10.1518/001872095779049543

Etzion, O. y Niblett, P. (2010). *Event processing in action*. Manning.

Expansión Política. (2025, 29 de enero). *Cuauhtémoc, GAM e Iztapalapa, con más fallas en cámaras y botones de pánico*. https://politica.expansion.mx/cdmx/2025/01/29/cuauhtemoc-gam-e-iztapalapa-con-mas-fallas-en-camaras-y-botones-de-panico

Home Office. (s. f.). *National ANPR Service (NAS): Rights of data subjects notice (section 44, Data Protection Act 2018)*. GOV.UK. https://www.gov.uk/government/publications/national-anpr-service-nas-data-subject-rights

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Kreps, J., Narkhede, N. y Rao, J. (2011, 12 de junio). *Kafka: A distributed messaging system for log processing* [Ponencia]. NetDB'11, Atenas, Grecia. https://www.microsoft.com/en-us/research/wp-content/uploads/2017/09/Kafka.pdf

Luckham, D. C. (2002). *The power of events: An introduction to complex event processing in distributed enterprise systems*. Addison-Wesley.

Nygard, M. (2011, 15 de noviembre). Documenting architecture decisions. *Cognitect Blog*. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Parlamento Europeo y Consejo de la Unión Europea. (2016). Reglamento (UE) 2016/679 relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (Reglamento general de protección de datos). *Diario Oficial de la Unión Europea*, *L 119*, 1–88. https://eur-lex.europa.eu/eli/reg/2016/679/oj

Stonebraker, M., Çetintemel, U. y Zdonik, S. (2005). The 8 requirements of real-time stream processing. *ACM SIGMOD Record*, *34*(4), 42–47. https://doi.org/10.1145/1107499.1107504

The Apache Software Foundation. (s. f.). *Apache Kafka documentation*. https://kafka.apache.org/documentation/

Timescale. (s. f.). *TimescaleDB documentation*. https://docs.timescale.com/

U.S. Environmental Protection Agency. (s. f.). *Rio operations center*. https://archive.epa.gov/international/jius/web/html/rio_operations_center.html

Anexos

Anexo A. Architecture Decision Records. Documento independiente que consolida los diez ADRs (archivo *Anexo-A-ADRs.docx*), conforme a la sección 4.2 del enunciado.

Anexo B. Configuración técnica y manuales. El repositorio del proyecto contiene la configuración de servicios y contenedores (archivos Docker Compose por nodo y archivos de configuración de cada servicio), el script centinela.cmd para instalar y operar todo desde Windows, los scripts de simulación de eventos, la documentación del sistema de correlación, el manual de operación con el guion de la puesta en simulación y la guía de despliegue en varias máquinas.